

使用 Stackdriver Logging 和 Stackdriver Trace for Cloud Functions

來源：<https://codelabs.developers.google.com/codelabs/cloud-function-logs-traces?hl=zh-tw>

步驟數：9

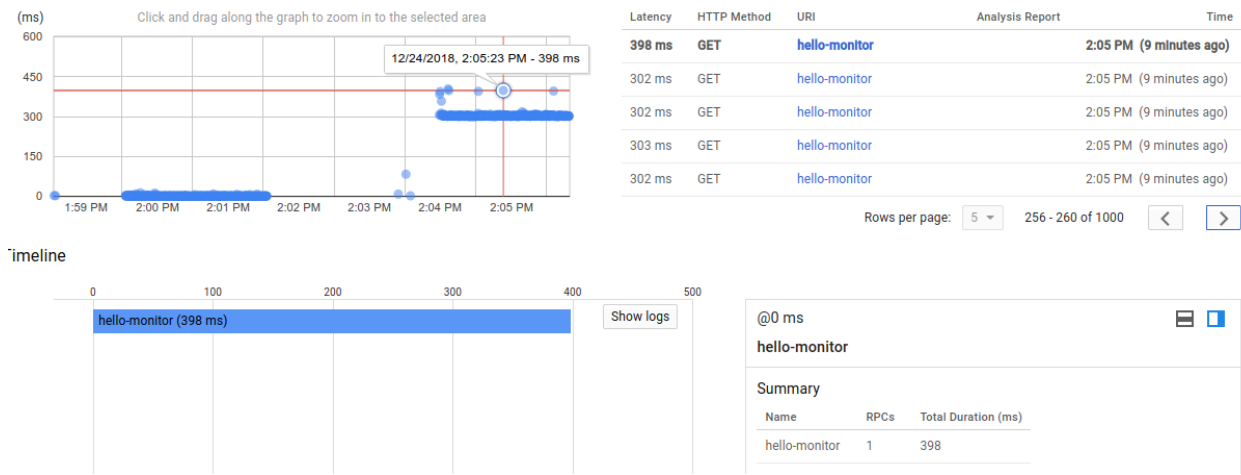
瞭解如何使用 Cloud Functions 的 Stackdriver Logging 和 Stackdriver Trace。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 部署簡易的 Cloud 函式](#)
4. [4. 使用負載產生器測試 Cloud Function 並傳送流量](#)
5. [5. 瀏覽記錄](#)
6. [6. 更新函式](#)
7. [7. 追蹤更新後的 Cloud 函式](#)
8. [8. 現在來清除資源](#)
9. [9. 後續步驟](#)

1. 簡介

在本程式碼研究室中，您將瞭解如何運用記錄和監控工具，這些工具適用於所有使用 Cloud Functions 的開發人員。無論您使用哪種支援的語言部署 Cloud Functions，都會隨附這些工具，協助您編寫及運作無伺服器程式碼，進而提高生產力。



您會使用 HTTP 觸發的 Cloud Functions，但涵蓋的所有內容也適用於其他語言，以及由其他事件觸發的 Cloud Functions。

步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。(如果沒有 Gmail 或 G Suite 帳戶，請先[建立帳戶](#))。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：console.cloud.google.com。

The screenshot shows the Google Cloud Platform interface for creating a new project. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown. Below this, the 'Select a project' section includes a search bar and a 'NEW PROJECT' button. The 'New Project' form has three main fields: 'Project name *' with the value 'my-kubernetes-codelab', 'Project ID: my-kubernetes-codelab. It cannot be changed later.' (with an 'EDIT' link), and 'Location *' with the value 'No organization' (with a 'BROWSE' link). At the bottom, there are 'CREATE' and 'CANCEL' buttons.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 G Suite 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

Cloud Shell

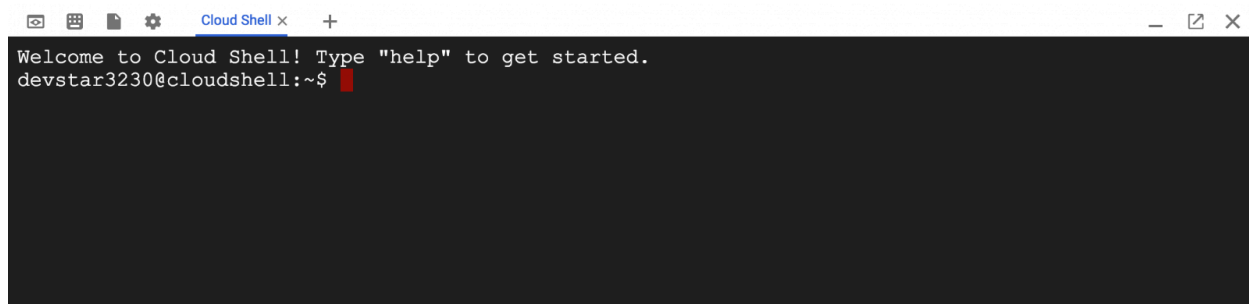
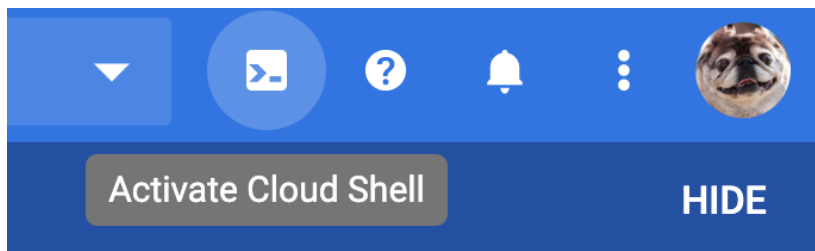
雖然可以從筆電遠端使用 Cloud Functions 及其記錄和監控功能，但您會使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

1. 如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」



即可 (佈建並連線至環境的作業需要一些時間才能完成)。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

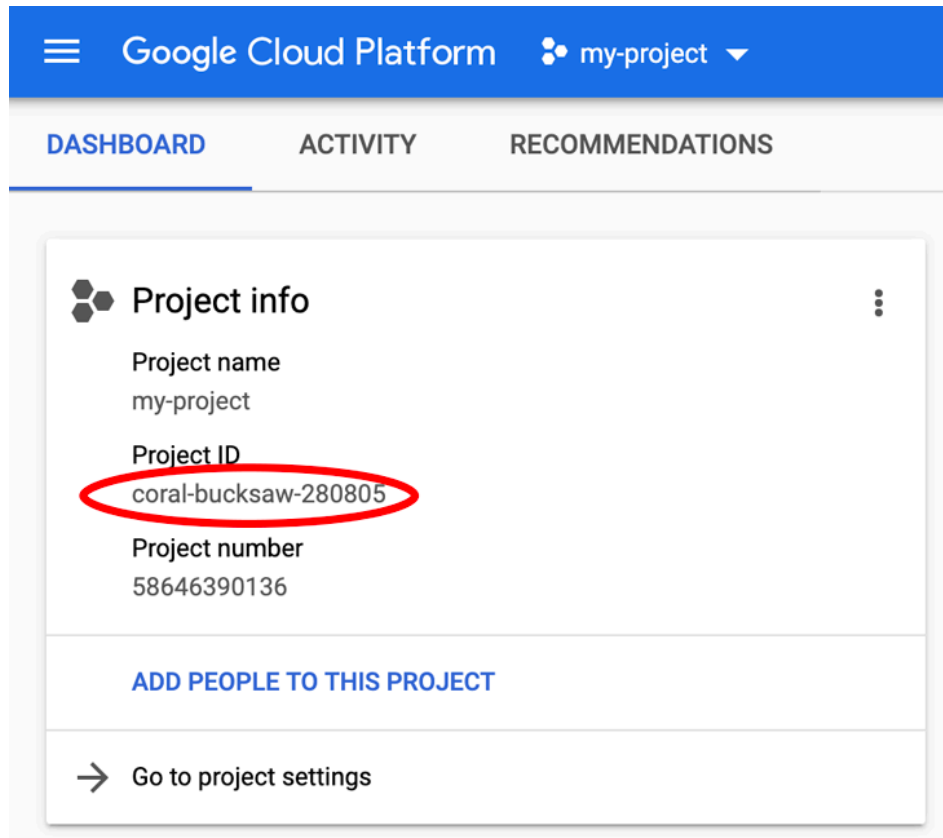
指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：



Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

2. 最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

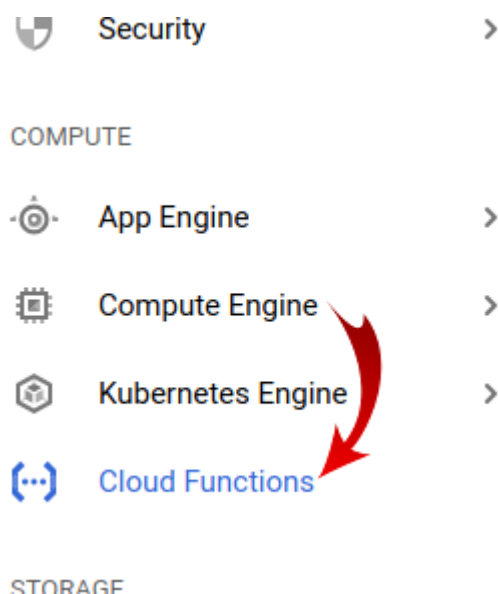
你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

步驟 3 · 約 2 分鐘

3. 部署簡易的 Cloud 函式

為了監控某些項目，請建立「Hello, World」Cloud 函式。在 Google Cloud 控制台的左選單中，按一下「Cloud Functions」，然後點選「建立函式」。



輸入「hello-monitor」做為新 Cloud Function 的名稱。

A screenshot of the 'Create function' form in the Google Cloud console. The form has a header with the Cloud Functions icon and a 'Create function' button. Below the header, there are four sections: 'Name' with a question mark icon and a text input field containing 'hello-monitor'; 'Memory allocated' with a dropdown menu showing '256 MB'; 'Trigger' with a dropdown menu showing 'HTTP'; and 'URL' with a text input field containing 'https://us-central1-my-project.cloudfunctions.net/hello-monitor'.

保留原始碼的所有預設值。(不過，您也可以選擇其他語言/執行階段)。

Source code

- ☒ Inline editor
- ☐ ZIP upload
- ☐ ZIP from Cloud Storage
- ☐ Cloud Source repository

Runtime

Node.js 6

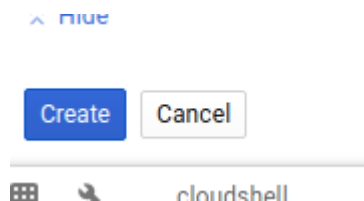
index.js package.json

```
1 /**
2  * Responds to any HTTP request.
3  *
4  * @param {!express:Request} req HTTP request context.
5  * @param {!express:Response} res HTTP response context.
6  */
7 exports.helloWorld = (req, res) => {
8   let message = req.query.message || req.body.message || 'Hello World';
9   res.status(200).send(message);
10 };
11
```

Function to execute ?

helloWorld

最後點選「建立」。



您應該會看到 Cloud 函式列在旁邊，並標有綠色勾號，表示函式已可叫用。

Cloud Functions		Overview		+ CREATE FUNCTION	REFRESH	DELETE	COPY
Filter functions						Columns	
☐ Name ^	Region	Trigger	Runtime	Memory allocated	Executed function	Last deployed	
☒ hello-monitor	us-central1	HTTP	Node.js 6	256 MB	helloWorld	12/24/18, 10:34 AM	

步驟 4 · 約 4 分鐘

4. 使用負載產生器測試 Cloud Function 並傳送流量

Cloud Function 部署完成後，請從指令列進行測試。

首先，請使用 Cloud Shell 發出下列指令：

```
$ gcloud functions describe hello-monitor
```

這應該會傳回 Cloud Function 的說明，包括 `httpsTrigger` 的網址，也就是用來叫用 Cloud Function 的 HTTP(S) 端點。網址應如下所示：`https://<region>-<project-id>.cloudfunctions.net/hello-monitor`。

現在只要在該網址上使用 `curl` 指令，即可觸發 Cloud Function。

```
$ curl https://<region>-<project-id>.cloudfunctions.net/hello-monitor
Hello World!
```

現在請使用簡單的 HTTP 負載測試工具 [Vegeta](#)。如要安裝，請在 Cloud Shell 中輸入下列指令：

```
$ go get -u github.com/tsenart/vegeta
```

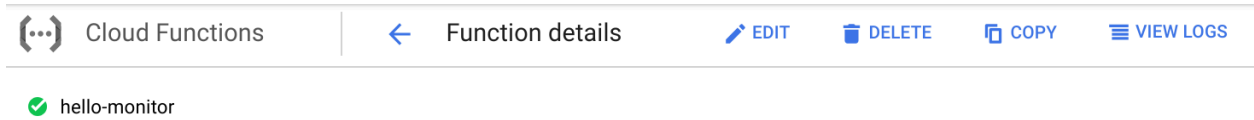
如要將部分流量傳送至 Cloud 函式 (每秒五項要求，持續幾分鐘)，請使用下列指令：

```
$ echo "GET https://<region>-<project-id>.cloudfunctions.net/hello-monitor" \  
| vegeta attack -rate=5 -duration=120s \  
> results.bin
```

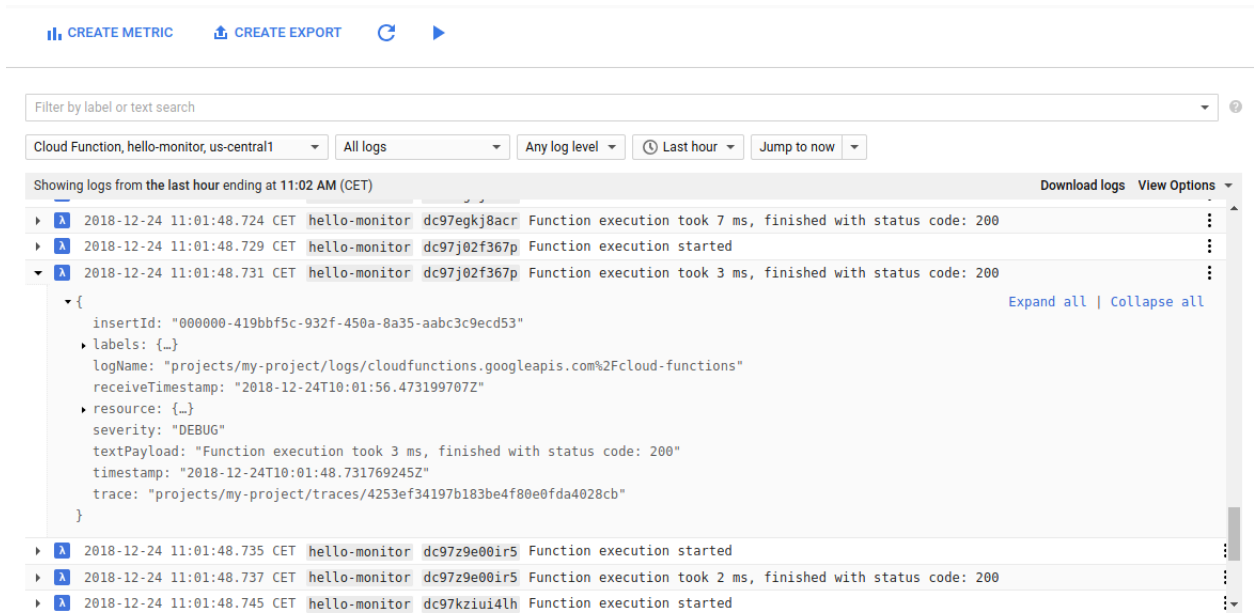
步驟 5 · 約 4 分鐘

5. 瀏覽記錄

在 Cloud Function 詳細資料檢視畫面中，按一下「查看記錄」。



這會將您帶往專案的 Stackdriver Logging 區段，只顯示 Cloud Functions 記錄。



對 Cloud Function 的所有要求都應傳回 200 狀態碼。

查看記錄時，您可以執行下列操作：

- 依記錄層級篩選 (在本例中，所有記錄都是 `debug` 層級)。
- 選取特定時間範圍 (相對或絕對)。
- 啟用記錄串流 (畫面頂端會顯示「Play」



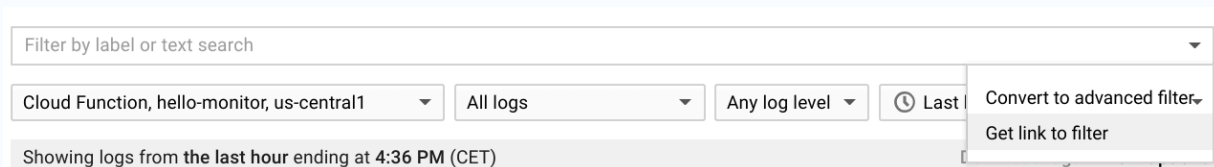
)。

- 複製記錄項目的連結 (與團隊成員分享)。
- 在資源內容中顯示記錄檔項目。
- 釘選記錄項目 (做為視覺提示)。
- 將記錄匯出至 BigQuery、Cloud Storage 或 Pub/Sub (或直接下載為 JSON 或 CSV 檔案)。

注意：在 Cloud Functions 程式碼中，標準輸出記錄會顯示 `__INFO__` 記錄層級，標準錯誤記錄則會顯示 `__ERROR__` 記錄層級。內部系統訊息具有 `__DEBUG__` 記錄層級。

注意：您可以使用左上方的選取器，依函式篩選記錄。您也可以在視窗頂端的搜尋列中搜尋標籤或執行全文搜尋。在本例中，標籤是 `hello-monitor` (函式名稱) 和每項要求的執行 ID。

您也可以複製篩選器並共用 (請查看搜尋方塊中的下拉式選單)。

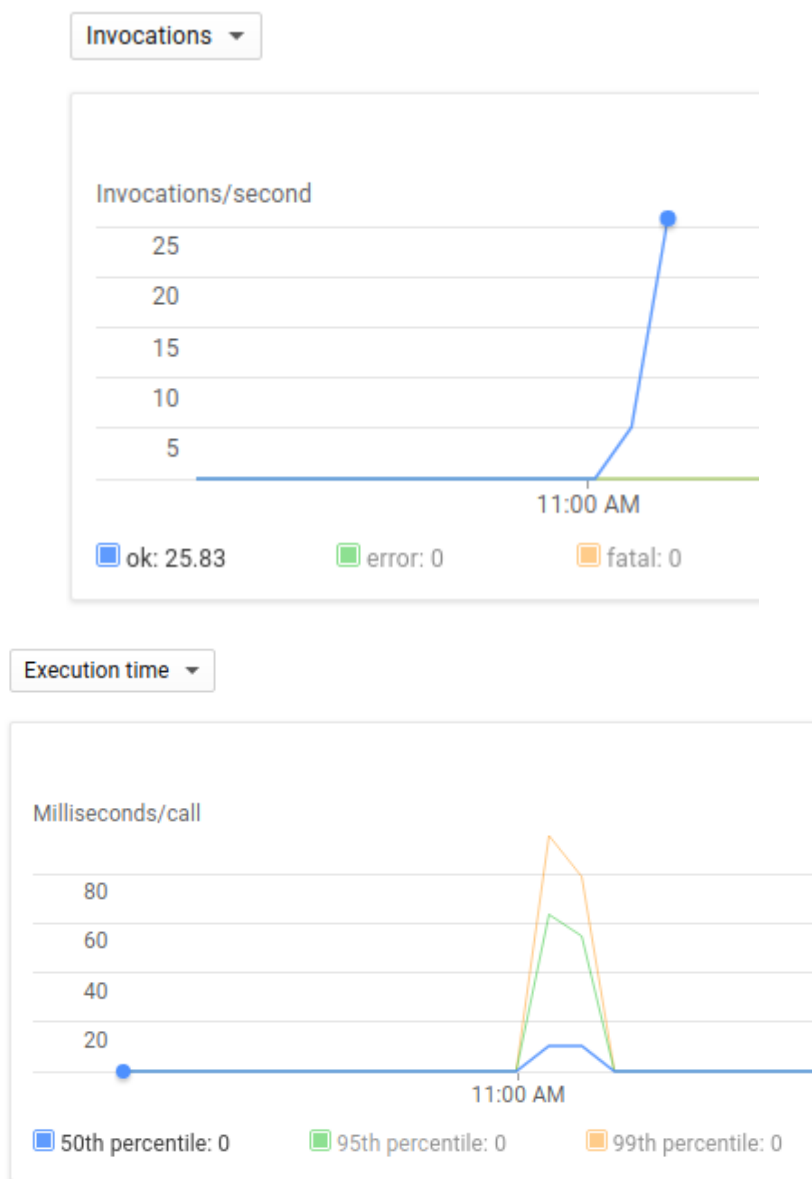


注意：除了上述圖形化工具，您也可以使用 `gcloud functions logs read` 存取 Cloud Functions 記錄，或透過 [這裡](#) 記錄的 API 存取。

步驟 6 · 約 4 分鐘

6. 更新函式

使用 Cloud 控制台前往「Function details」(函式詳細資料) 檢視畫面，觀察您使用負載測試工具建立的尖峰，以及每秒的叫用次數和執行時間。



如要更詳細地觀察延遲和 RPC 呼叫，可以使用 Stackdriver Trace，但您必須先對 Cloud Functions 進行幾項變更。請完成下列步驟：

1. 將救命的 `node-emoji` 套件新增為依附元件。
2. 更新函式程式碼，使用 `node-emoji` 模組並導入一些延遲。
3. 新增環境變數，為 Cloud Functions 啟用 Stackdriver Trace。

在「函式詳細資料」中，按一下「編輯」即可修改函式。



編輯 `package.json` 檔案，為 `node-emoji` 套件新增依附元件。

```
{
  "name": "sample-http",
  "version": "0.0.1",
  "dependencies": {
    "node-emoji": "^1.8.1"
  }
}
```

將 `index.js` 的內容變更為下列內容，即可編輯實際函式：

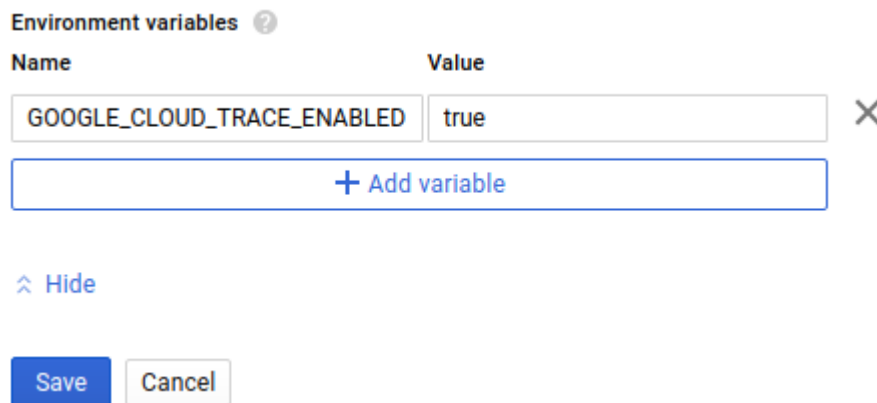
```
const emoji = require('node-emoji');

exports.helloWorld = (req, res) => {
  let message = req.query.message || req.body.message || 'Hello World!';

  // add some latency, functions are too quick!
  setTimeout(function() {
    message += emoji.random().emoji;
    res.status(200).send(message);
  }, (3 * 100)); // 300ms
};
```

這會在 Cloud 函式暫停 300 毫秒後，將隨機表情符號新增至傳回的訊息。

最後，新增名為 `GOOGLE_CLOUD_TRACE_ENABLED` 的 Cloud Functions 環境變數，並按照下列方式設為 `true`：



Name	Value
GOOGLE_CLOUD_TRACE_ENABLED	true

[+ Add variable](#)

[^ Hide](#)

[Save](#) [Cancel](#)

按一下 [儲存]。

返回 Cloud Shell，並回想一下在剛部署的 Cloud 函式上產生負載的指令：

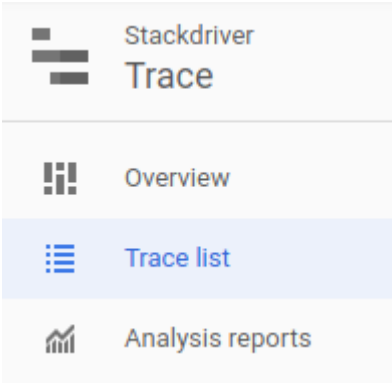
```
$ echo "GET https://<region>-<project-id>.cloudfunctions.net/hello-monitor" \
  | vegeta attack -rate=5 -duration=120s \
  > results.bin
```

現在您已準備好觀察產生的追蹤記錄清單，不需要其他設定，也不必在程式碼中使用特定追蹤記錄程式庫！

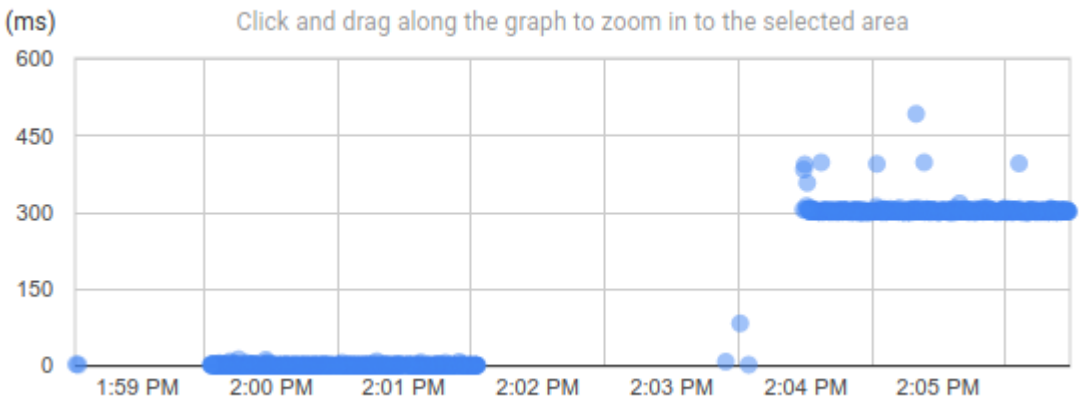
步驟 7 · 約 4 分鐘

7. 追蹤更新後的 Cloud 函式

使用左側選單前往「追蹤記錄清單」(位於「Stackdriver Trace」下方)。

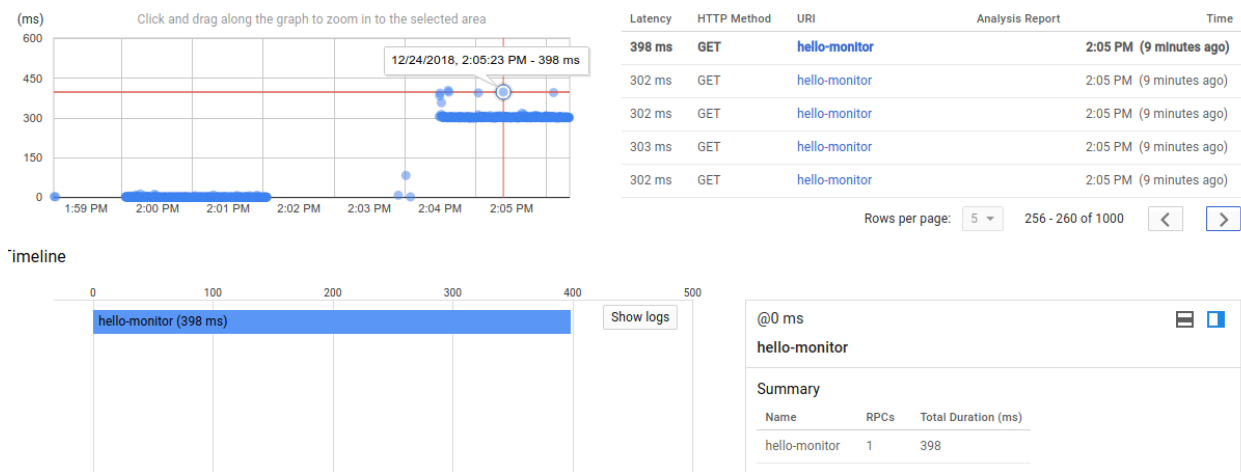


您應該會看到類似下圖的畫面：

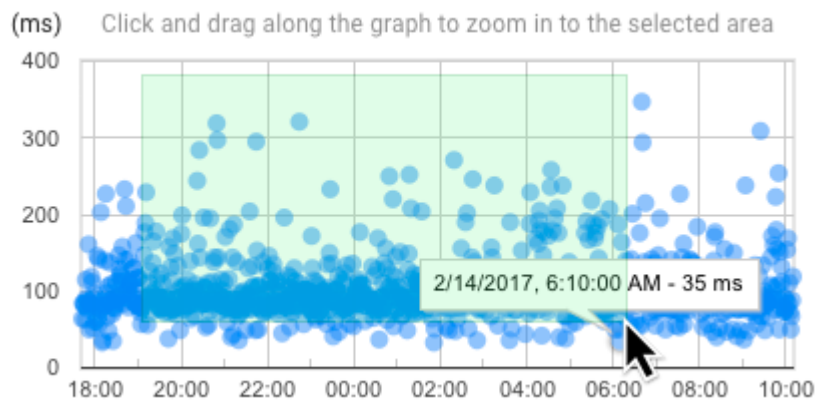


這應該會清楚顯示，Cloud Function 導入的延遲時間確實為 300 毫秒。

圖表上的每個點都是一項要求，您可以查看詳細資訊，例如時間戳記、HTTP 方法和狀態、標籤、對應記錄項目的連結，以及 Cloud Functions 進行的任何後續 RPC 呼叫。



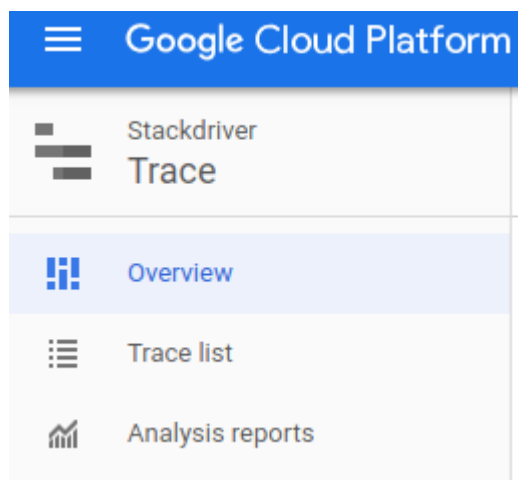
若要放大圖表，只要點選並拖曳即可。



如要縮小，請按一下頁面頂端的「取消縮放」。

由於您部署了單一 Cloud Function，圖表只會顯示 `hello-monitor` URI 上的 `GET` 要求，但您可以依據 HTTP 方法 (GET、POST、DELETE)、HTTP 狀態 (2XX、3XX) 或使用要求篩選器來篩選追蹤記錄。

前往左選單中的「總覽」：



您可以在這個總覽頁面中查看近期追蹤記錄和其他深入分析資訊。

Recent traces		
LATENCY	URI	TIME
13 ms	hello-monitor	12:1...
53 ms	hello-monitor	12:0...
30 ms	hello-monitor	12:1...
18 ms	hello-monitor	12:0...
23 ms	hello-monitor	12:0...
1 ms	hello-monitor	12:5...
See more ...		

您也可以根據 URI 要求篩選器、HTTP 方法、HTTP 狀態和時間範圍的組合，建立 **自訂報表**。您甚至可以將產生的值與時間基準線進行比較。

New report request

Request Filter

☐ Only analyze requests that result in remote procedure calls 

HTTP method

All

HTTP status

All

Report name (optional)

Time range

Custom

Start

Dec 24, 2018, 1:22:30 PM CET

End

Dec 24, 2018, 2:22:30 PM CET

☐ Compare to baseline

Submit

Cancel

如果您設法設定正確的時間範圍並取得足夠的資料點，就能產生報表，顯示初始 Cloud 函式和新函式之間的重要延遲變化。

Custom reports

Filter by

☐ Name	Latency shift 	Created 	Status
☐ hello-monitor @ 2018-12-24T04:33:58.883-08:00	MAJOR CHANGE	2018-12-24 (14:38:30)	 Succeeded
☐ hello-monitor @ 2018-12-24T04:34:48.701-08:00		2018-12-24 (14:34:48)	 Succeeded

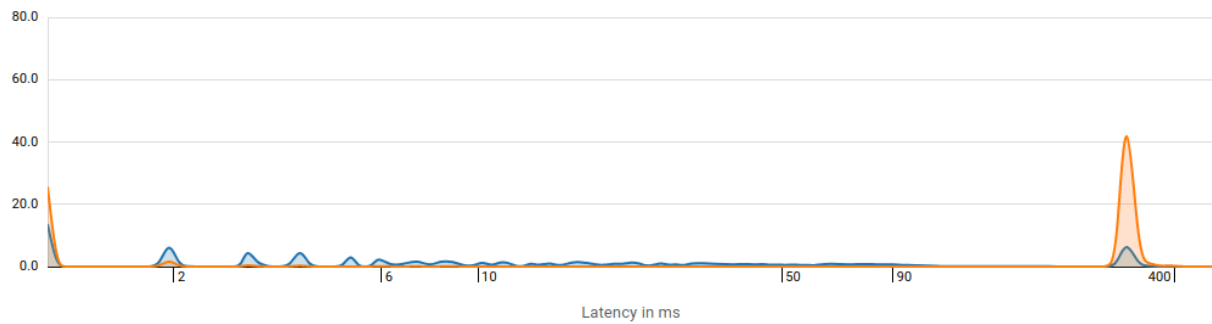
Report Name	URI	HTTP method	Status code	Report Creation Time
hello-monitor @ 2018-12-24T04:33:58.883-08:00	hello-monitor	GET	200	2018-12-24 (14:38:30)

hello-monitor MAJOR CHANGE   

[Density distribution](#) [Cumulative distribution](#)

Overall latency for requests

% of total requests



	Start 2018-12-24 (12:00:04)	End 2018-12-24 (14:25:04)		Start 2018-12-24 (13:33:58)	End 2018-12-24 (14:33:59)
---	--------------------------------	------------------------------	---	--------------------------------	------------------------------

Latency

Request percentile	 Latency	Sample Traces	 Latency	Sample Traces
25%	2 ms	1 2 3 4	1 ms  -50%	1 2 3 4
50%	8 ms	1 2 3 4	300 ms  +3,650%	1 2 3 4
90%	299 ms	1 2 3 4	305 ms  +2%	1 2 3 4
95%	301 ms	1 2 3 4	308 ms  +2%	1 2 3 4
98%	304 ms	1 2 3 4	339 ms  +12%	1 2 3 4
99%	306 ms	1 2 3 4	343 ms  +12%	1 2 3 4

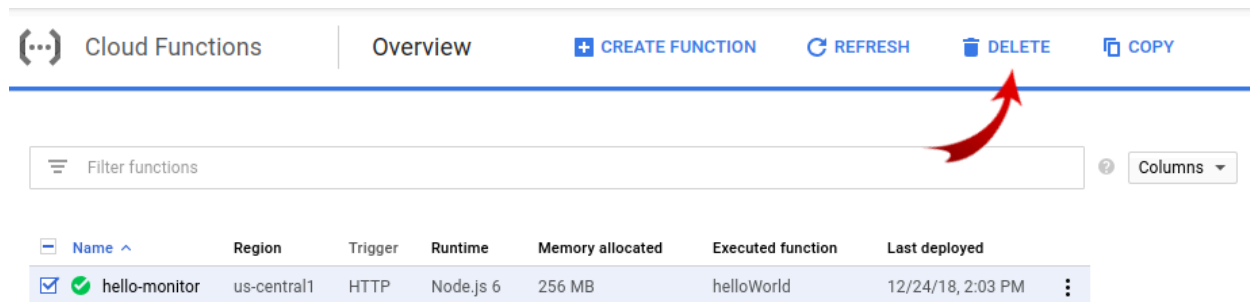
這類自訂報表可用於找出效能問題的發生時間，以及追蹤服務等級指標 (SLI)，例如使用者要求延遲時間。

步驟 8 · 約 2 分鐘

8. 現在來清除資源

本程式碼研究室到此結束！

Cloud Functions 和 Stackdriver 工具都是無伺服器平台，閒置時不會產生費用，但請發揮雲端公民精神，刪除 Cloud Function。只要在「Cloud Functions」下方的「Overview」中選取 `hello-monitor`，然後按一下「Delete」即可。



步驟 9 · 約 1 分鐘

9. 後續步驟

以下提供一些後續閱讀資料：

- [Stackdriver Monitoring 記錄指標](#)
- [Stackdriver Error Reporting](#)

/
